CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 5: C Wrapup, Floating Point

Instructor: Justin Yokota

Floating Point System: V1

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- This represents the number $(-1)^{S} * 0bM.MMM * 2^{(0bXXX+(-3))}$

Floating Point V1: Example

- For this example, let's assume we're working with a system with 3 exponent bits (bias of -3) and 4 mantissa bits.
- Convert 2.75 to a V1 float
- Step 1: Write the number in binary
  - 0b10.11 = 0b1.011000… * 2^1
- Step 2: Determine Sign/Exponent/Mantissa
  - Sign: Positive → 0
  - Exponent: 1-(-3) = 4 → 0b100
  - Mantissa: 0b1011
- Step 3: Concatenate
  - 0b0 100 1011
  - 0x4B

Floating Point V1: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0xC3CC0000 as a V1 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1000 0111 → 135-127 = 8
  - Mantissa: 0b1001 1000… → 0b1.001 1000 = $1+2^{-3}+2^{-4}$
  - $(1+2^{-3}+2^{-4})*2^{8} = 2^{8}+2^{5}+2^{4}=304$
- Convert 0x00000000 as a V1 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 → 0-127 = -127
  - Mantissa: 0b0000 0000 0000 0000 0000 000 → 0
  - $0*2^{-127}=0$

Agenda

- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard
- Intro to Assembly Languages
- RISC-V Programming Paradigm
  - `add` and `sub`
  - Immediates: The `addi` instruction
- Demo: Venus

Agenda

- **Floating Point**
  - Simplified Model
  - **Optimizations**
  - IEEE-754 Standard
- Intro to Assembly Languages
- RISC-V Programming Paradigm
  - `add` and `sub`
  - Immediates: The `addi` instruction
- Demo: Venus

Optimization 1: The implicit 1

- Note that our mantissa is guaranteed to not have any leading zeros
  - If we wanted to write $0.234*10^5$, we'd instead write it as $2.340*10^4$
- In binary, every digit is only either 1 or 0. Since the MSB can't be 0, it must therefore be 1.
  - If the first bit will always be 1, we don't need to store it!
- Therefore, we can save 1 bit (or alternatively add another bit of precision) to the mantissa by not including the MSB of our mantissa
  - This is known as the implicit 1, and the resulting mantissa is a "normalized" number.

Floating Point System: V2

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- This represents the number $(-1)^{S} * 0b1.MMMM * 2^{(0bXXX+(-3))}$

Floating Point V2: Example

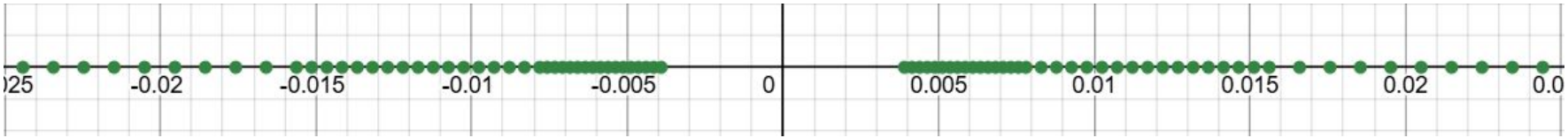
- For this example, let's assume we're working with a system with 3 exponent bits (bias of -3) and 4 mantissa bits.
- Convert 2.75 to a V1 float
- Step 1: Write the number in binary
  - 0b10.11 = 0b1.011000… * 2^1
- Step 2: Determine Sign/Exponent/Mantissa
  - Sign: Positive → 0
  - Exponent: 1-(-3) = 4 → 0b100
  - Mantissa: 0b0110
- Step 3: Concatenate
  - 0b0 100 0110
  - 0x46

Floating Point V2: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0x00000000 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 -> 0-127 = -127
  - Mantissa: 0b0000… → 0b1.000…
  - $1*2^{-127}$
- Convert 0x00000001 as a V2 float to decimal
  - Sign: 0 → Positive
  - Exponent: 0b0000 0000 → 0-127 = -127
  - Mantissa: 0b0000…1 → 0b1.000…1 = $1+2^{-23}$
  - $(1+2^{-23})*2^{-127}=2^{-127}+2^{-150}$

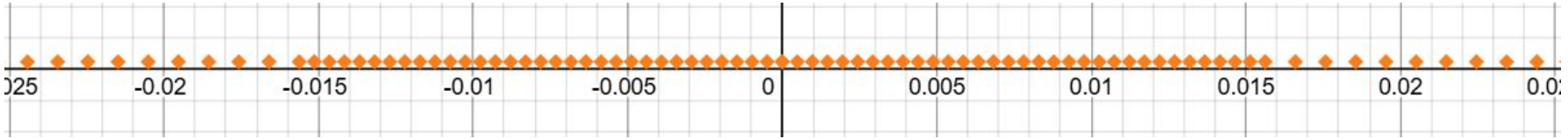
Problems with the implicit 1: Underflow

- The smallest number (in absolute value) we can represent is 2^{-127}
  - We can't represent 0
- The second smallest number is $2^{-127}+2^{-150}$
  - There's a big gap in the set of representable numbers (10 million times bigger than the gaps between consecutive numbers)
- This is known as an **underflow**; the result of computation gets too small to be represented.
- In the below diagram, each dot is a representable number (3 exponent, 4 mantissa bits).



Solution: Denormalized numbers

- Solution: Change the behavior of exponent 0b000…0 so that its range extends to 0
- How to do that?
  - If we use the same step size as exponent 0b000…1, then we can get exactly to 0
  - When dealing with these numbers, use an implicit 0 instead of an implicit 1, so it doesn't overlap with exponent 0b000…1
- Ends up losing precision at small numbers (so there's still underflow), but at least it's not a sudden cliff drop.



Floating Point System: V3

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- If the exponent bits are nonzero, then it represents the number $(-1)^S * 0b1.MMMM * 2^{(0bXXX+(-3))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S * 0b0.MMMM * 2^{(0b000+(-3)+1)}$ (also known as a denormalized number, or "denorm")

Floating Point V3: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0x00000000 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000. All zeros, so exponent meaning is 0-127+1 = -126
  - Mantissa: 0b0000… -> 0b0.000…
  - $0*2^{-126} = 0$ (We can represent 0!)
- Convert 0x00000001 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 -> 0-127+1 = -126
  - Mantissa: 0b0000…1 -> 0b0.000…1 = $0+2^{-23}$
  - $(0+2^{-23})*2^{-126}=2^{-149}$ (Much closer to 0)

Optimization 2: Dealing with infinity, division by zero

- We get a fairly large range using this (10^{38} with 8 exponent bits), but we can still exceed the maximum float.
- We also should deal with “1 / 0”
- Ideally, we’d like to include an “infinity” value to handle these cases
- While we’re at it, let’s add some values for error handling, that don’t correspond to any actual number
- What bitstrings do we use for this?
    - Since we want an infinity, let’s use the largest exponent for this

Floating Point System: Final Version

- An IEEE-754 standard binary floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- If the exponent bits are nonzero and not all ones, then it represents the number $(-1)^S$*0b1.MMMM * $2^{(0bXXX+(-3))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S$*0b0.MMMM * $2^{(0b000+(-3)+1)}$
- If the exponent bits are all ones, then:
  - If the mantissa is all zeros, it's either ∞ or -∞ (depending on the sign bit)
  - If the mantissa isn't all zeros, then it's a NaN (which means "Not a Number")

Floating Point System: Final Version

- An IEEE-754 standard binary floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 8 exponent bits (bias of -127) and 23 mantissa bits

SXXX XXXX XMMM MMMM MMMM MMMM MMMM MMMM

- If the exponent bits are nonzero and not all ones, then it represents the number $(-1)^S$ * 0b1.MMMM… * $2^{(0bXXXX\ XXXX+(-127))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S$*0b0.MMMM… * $2^{(0b0+(-127)+1)}$
- If the exponent bits are all ones, then:
  - If the mantissa is all zeros, it's either ∞ or -∞ (depending on the sign bit)
  - If the mantissa isn't all zeros, then it's a NaN (which means "Not a Number")

Floating Point Final: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0xFF80 0000 as an IEEE-754 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1111 1111. All ones, so we're dealing with a special case
  - Mantissa: 0b0000… -> All zeros
  - -∞
- Convert 0xFF80 0001 as an IEEE-754 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1111 1111. All ones, so we're dealing with a special case
  - Mantissa: 0b0000…1 -> Not all zeros
  - NaN

Agenda

- **Floating Point**
  - Simplified Model
  - Optimizations
  - **IEEE-754 Standard**
- Intro to Assembly Languages
- RISC-V Programming Paradigm
  - `add` and `sub`
  - Immediates: The `addi` instruction
- Demo: Venus

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats

IEEE-754: Interchange Formats

- IEEE-754 specifies certain combinations of exponent and significand bits with special names:
- Single precision (also known as float):
  - 8 exponent bits, -127 bias, 23 mantissa bits
- Double precision (also known as double):
  - 11 exponent bits, -1023 bias, 52 mantissa bits
- Quad precision:
  - 15 exponent bits, 112 mantissa bits
- Octuple precision:
  - 19 exponent bits, 236 mantissa bits
- Half precision:
  - 5 exponent bits, 10 mantissa bits

IEEE-754: Interchange Formats

- C's float and double correspond to single-precision and double-precision floating point numbers, respectively
- Float:
  - Has enough precision for 6-7 decimal digits
  - Max value around 10^{38}
- Double:
  - Has enough precision for ~13 decimal digits
  - Max value around 10^{308}
  - Despite its name, it is generally considered the "default" floating point representation
  - You should almost always use a double; the precision loss is way more costly than a few bytes

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations
- Exception Handling (See skipped slides)

IEEE-754: Rounding Rules

- Several different options, but the most common one is “round to the nearest value, and break ties to the even number (last bit 0)
  - Ex. To round 14.5 to the nearest 2, go to 14, since 14 is closer to 14.5 than 16.
  - Ex. To round 15 to the nearest 2, go to 16, since 16 ends in more 0 bits than 14
- This is intended to be deterministic, but in practice, it’s rather unpredictable
- Generally a good idea not to rely too much on rounding behavior.
- Rounding happens after every operator in a string of operations, so we get slightly less precise results every step
  - This is why we use so many bits for the mantissa in doubles, and why doubles are the standard float option; precision is more important than range
  - Often different orders of doing operations yields different results even if they should be mathematically equivalent.

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations

IEEE-754: Operations

- Requires conversion to/from integer, arithmetic, comparisons, abs/negate, floor/ceiling, etc.
  - Two new operators are: square root and fused multiply-add (a*b+c done in one step)
- Additionally recommends native support for other math operations (e^x, 2^x, 10^x, log x, distance formula, trig functions, hypertrig functions, arctrig functions)
- Most of these functions are available through the <math.h> library

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations
- Exception Handling

IEEE-754: Exception Handling

- Defines five classes of exceptions, along with recommended behavior:
- Invalid operation
  - Ex. sqrt(-1.0)
  - By default, returns a NaN (quiet)
- Division by zero
  - By default, returns infinity
- Overflow
  - By default, returns infinity
- Underflow
  - Returns a denorm. Note that we consider any result using a denorm to suffer from underflow (due to precision loss)
- Inexact value
  - Any math that yields a number that can’t be exactly represented, like 1.0/3
  - Rounds to a representable number according to the rounding rule.

For what positive integers n does (1.0/n)+(1.0/n)+... (n times) == 1.0?

0

All n **(A)**

Most n **(B)**

A few n **(C)**

No n **(D)**

For what positive integers n does (1.0/n)+(1.0/n)+... (n times) == 1.0?

0

(A) All n

0%

(B) Most n

0%

(C) A few n

0%

(D) No n

0%

For what positive integers n does (1.0/n)+(1.0/n)+... (n times) == 1.0?

0

(A) All n

0%

(B) Most n

0%

(C) A few n

0%

(D) No n

0%

Agenda

- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard
- **Intro to Assembly Languages**
- RISC-V Programming Paradigm
  - `add` and `sub`
  - Immediates: The `addi` instruction
- Demo: Venus

Building a computer from the ground up

- Any program we write needs to run on a circuit in order to be useful.
- However, circuit-level programming is highly restrictive.
  - With C, a human designed the programming language
  - With a circuit, silicon dictates what language we can design.
- Early computers essentially had to be rebuilt for every program you wanted to run, since different computations required different circuits.
- One major advancement in computer science was the creation of software-based languages.
  - Instead of making a new circuit for every problem you want to solve, make a circuit (called a CPU) that solves the problem “Carry out a sequence of instructions stored as binary data”. Then solve your problem by writing instructions in binary data.

Assembly Language

- We don’t want to change the CPU after we build it, so when designing our CPU, we need to decide on a specific set of instructions that will be supported by the CPU, along with a way to translate each instruction to a binary form.
- Different CPUs implement different sets of instructions. The set of instructions a particular CPU implements is an Instruction Set Architecture (ISA), and the programming language defined by the ISA is commonly known as an assembly language.
    - Examples: ARM (cell phones), Intel x86 (i9, i7, i5, i3), IBM/Motorola PowerPC (old Macs), MIPS, RISC-V, ...

Assembly Language

- C is generally considered a “lower-level” language than Java or Python, because it’s “closer” to the underlying CPU
  - Less is automated by the language, so you get faster runtimes, but have to keep track of more things
- Ultimately, C is still considered a high-level language:
  - C splits long lines of code into instructions for you
  - C sets up the stack for you
  - C lets you just call a function, and it works magically.
  - C lets you name variables, and will keep track of that variable.
- Once you start working with assembly languages, almost everything is the result of an explicit instruction by the programmer.

Instruction Set Architectures

- Early trend in ISA design was to add more and more instructions to new CPUs to do elaborate operations
  - VAX architecture had an instruction to multiply polynomials!
- RISC philosophy (Cocke IBM, Patterson, Hennessy, 1980s) – Reduced Instruction Set Computing
  - Keep the instruction set small and simple.
  - Let software do complicated operations by composing simpler ones.
  - A simpler CPU is easier to iterate on (allowing for faster development), and can generally be made faster than a complex CPU (we're often limited by the slowest instruction we decide to implement)

RISC-V

- For the purposes of this class, we'll be learning RISC-V as our assembly language
- Why?
  - RISC-V is relatively simple, in that there's only a few instructions in the base instruction set, and that instructions themselves follow a consistent format.
    - x86 is a more popular language (base CPU for most laptops/desktops), but is a CISC language that Huffman encodes its instructions
    - Project 3: Build a complete RISC-V CPU
  - RISC-V is relatively popular, open-source, and growing in popularity
  - RISC-V was invented in Berkeley in 2010.

RISC-V Resources

- CS 61C Reference Card
  - https://cs61c.org/sp24/pdfs/resources/reference-card.pdf
  - Lists out the entire base architecture
- Venus
  - https://venus.cs61c.org/
  - Online RISC-V simulator

Agenda

- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard
- Intro to Assembly Languages
- **RISC-V Programming Paradigm**
  - `add` and `sub`
  - Immediates: The `addi` instruction
- Demo: Venus

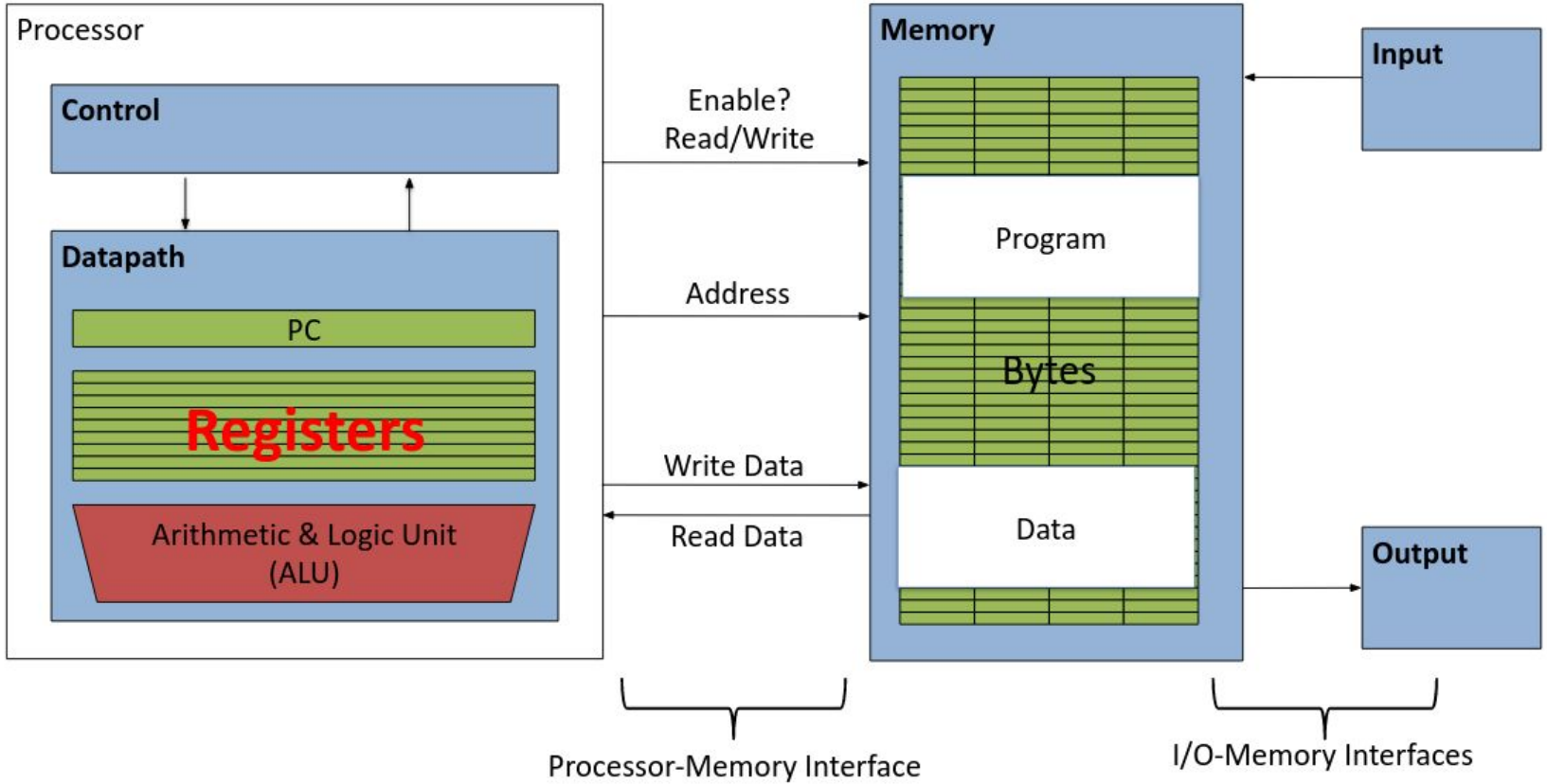
Overarching view of RISC-V

- A RISC-V system is composed of two main parts:
  - The CPU, which is responsible for computing
  - Main memory, which is responsible for long-term data storage (Monday)
- The CPU is designed to be extremely fast, often completing one instruction every nanosecond or faster
  - Note: Light travels 30 cm in 1 nanosecond. In other words, it takes longer for light to travel from one end of my laptop to the other, than it does for a CPU to finish one instruction.
- Going to main memory often takes hundreds or even thousands of times longer.
- The CPU can store a small amount of memory, through components called registers.

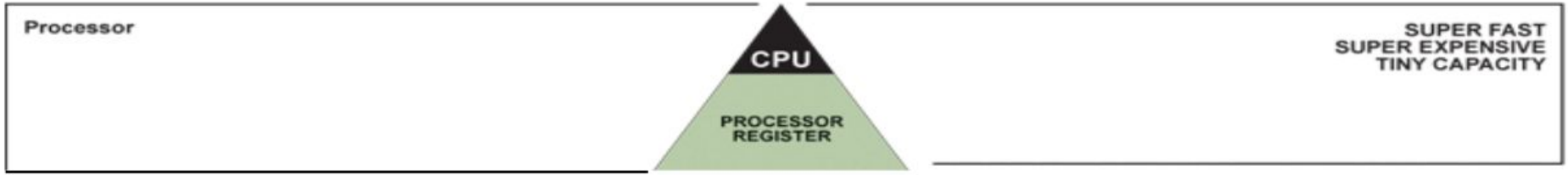
Registers

- A register is a CPU component specifically designed to store a small amount of data. Each register stores 32 bits of data (for a 32-bit system) or 64 bits of data (for a 64-bit system). For the purposes of this class, we consider RV32 only (which uses 32-bit registers)
- This data is purely binary; types do not exist at the assembly level
  - It’s the programmer’s responsibility to keep track of that register and its type.
- Registers are a hardware component, so once you make the CPU, you can’t change the number of registers available.
  - Can't "make" a new register when defining a new variable; have to delete an existing register for each variable you make.
- RISC-V gives access to 32 integer registers

Aside: Registers are Inside the Processor



Great Idea #3: Principle of Locality / Memory Hierarchy



Registers

- Registers are a hardware component, so once you make the CPU, you can't change the number of registers available.
- RISC-V gives access to 32 integer registers
- Registers are numbered from 0 to 31
  - Referred to by number: x0 – x31
- The register x0 is special and always stores 0 (More on this later). So only 31 registers are available to hold variables
- The other 31 registers are all identical in behavior; the only difference between different registers is the conventions we follow when using them.
- Later, we'll give them names to hint at what conventions get used on which registers.

Instructions

- Each line of RISC-V code is a single instruction, which executes a simple operation on registers.
- Instructions are generally written in the format
  - <instruction name> <destination register> <operands>
  - Ex. “`add x5 x6 x7`” means “Add the values stored in x6 and x7, and store the result in x5”
  - Commas can be added between registers (“`add x5, x6, x7`”), but this is optional.

Instructions

- Comments are written using the # symbol.
  - Anything written after a # on a line gets ignored
  - Similar to Python comment syntax
- Important: Comments are far more important in RISC-V than in other languages. In a higher level language, you can sometimes get away with choosing variable names so that the code is self-documenting. **In RISC-V, we don’t have variable names!**
- Uncommented RISC-V code is practically impossible to debug properly. If you don’t comment your code, we may not be able to help debug in office hours.

Addition and Subtraction of Integers (1/3)

Addition in Assembly

- Example: `add x1,x2,x3` (in RISC-V)
- Equivalent to: `a = b + c` (in C)
- where C variables ⇔ RISC-V registers are:

`a ⇔ x1, b ⇔ x2, c ⇔ x3`

Subtraction in Assembly

- Example: `sub x3,x4,x5` (in RISC-V)
- Equivalent to: `d = e - f` (in C)
- where C variables ⇔ RISC-V registers are:

`d ⇔ x3, e ⇔ x4, f ⇔ x5`

Addition and Subtraction of Integers (2/3)

How to do the following C statement?

```
a = b + c + d - e;
```

- Break into multiple instructions

```
add x10, x1, x2  # a_temp = b + c
add x10, x10, x3 # a_temp = a_temp + d
sub x10, x10, x4 # a = a_temp - e
```

Note: A single line of C may break up into several lines of RISC-V.

Note: Everything after the hash mark on each line is ignored (comments).

Addition and Subtraction of Integers (3/3)

How do we do this?

```
f = (g + h) - (i + j);
```

- Use intermediate temporary register

```
add x5, x20, x21 # a_temp = g + h
add x6, x22, x23 # b_temp = i + j
sub x19, x5, x6  # f = (g + h) - (i + j)
```

Note: By using x5 and x6 in this way, we overwrite any data that used to be in those registers.

Note: We could also have written this as `f = g + h - i - j;` which allows us to compute this without any temporary registers. A smart compiler may write code this way instead.

Immediates

- Immediates are numerical constants.
- They appear often in code, so there are special instructions for them.
- Add Immediate:
  - `addi x3,x4,10` (in RISC-V)
  - `f = g + 10` (in C)
  - where RISC-V registers `x3,x4` are associated with C variables `f,g`
- Syntax similar to add instruction, except that last argument is a number instead of a register.
- Common mistake: `addi x3,x4,x5` / `add x3,x4,10` are both invalid RISC-V instructions; be careful with using the register version vs the immediate version of an instruction!

Immediates

- There is no Subtract Immediate in RISC-V: Why?
  - There are add and sub, but no addi counterpart
- Limit types of operations that can be done to absolute minimum
  - if an operation can be decomposed into a simpler operation, don't include it
  - **`addi …, -X = subi …, X`** => so no subi

    `addi x3,x4,-10` (in RISC-V)

    `f = g - 10` (in C)
- where RISC-V registers **`x3,x4`** are associated with C variables **`f,g`**

Register Zero

- One particular immediate, the number zero (0), appears very often in code.
- So the register zero (**`x0`**) is ‘hard-wired’ to value 0; e.g.

  `add x3,x4,x0` (in RISC-V)

  `f=g` (in C)
- where RISC-V registers **`x3,x4`** are associated with C variables f, g
- Defined in hardware, so an instruction

  `add x0,x3,x4` will not do anything!

Agenda

- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard
- Intro to Assembly Languages
- RISC-V Programming Paradigm
  - `add` and `sub`
  - Immediates: The `addi` instruction
- **Demo: Venus**

Venus Demo

- Done in lecture
- Add 1+2+3+4+5

```
addi x5 x0 0
addi x6 x0 5
add x5 x5 x6
addi x6 x6 -1
add x5 x5 x6
addi x6 x6 -1
add x5 x5 x6
addi x6 x6 -1
add x5 x5 x6
addi x6 x6 -1
add x5 x5 x6
addi x6 x6 -1
```